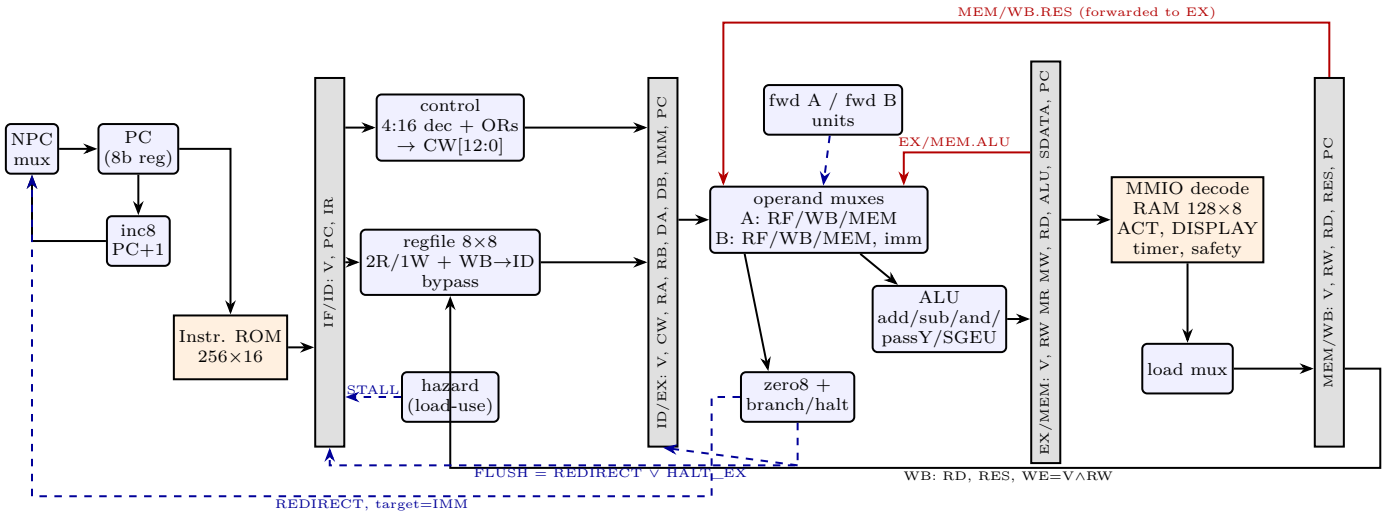
Architecture of the WM-8 Pipelined Microcontroller

ELL7282/ELL782 Introduction to Computer Architecture – Assignment 2

1. Block diagram



Grey bars: pipeline registers (built from Logisim Registers, one per field, all clocked by the single global clock, asynchronous `CLR=RESET`). Red: forwarding paths into EX. Dashed: hazard and control-transfer signals. Every multiplexer, decoder, comparator, adder, incrementer and the ALU is a gate-level sub-circuit (`mux2_8`, `mux4_8`, `mux8_8`, `dec3to8`, `dec4to16`, `eq3`, `eq8`, `zero8`, `fa1/add8`, `ha1/inc8`, `and8en`, `alu8`). The Plexers and Arithmetic libraries are not even loaded in the `.circ`.

2. Pipeline registers and control word

Register	Fields (Logisim register label: width)	bits
PC	`PC`:8 (enable `PC_EN`)	8
IF/ID	`IFID.V`:1, `IFID.PC`:8, `IFID.IR`:16 (common enable `IFID_EN`)	25
ID/EX	`IDEX.V`:1, `IDEX.CW`:13, `IDEX.RA`:3 (=destination), `IDEX.RB`:3, `IDEX.DA`:8, `IDEX.DB`:8, `IDEX.IMM`:8, `IDEX.PC`:8	52
EX/MEM	`EXMEM.V`:1, `EXMEM.CTRL`:3 (`RW,MR,MW`), `EXMEM.RD`:3, `EXMEM.ALU`:8 (result or address), `EXMEM.SDATA`:8 (forwarded store data), `EXMEM.PC`:8	31
MEM/WB	`MEMWB.V`:1, `MEMWB.RW`:1, `MEMWB.RD`:3, `MEMWB.RES`:8 (ALU or load data), `MEMWB.PC`:8	21
other	`HALTED`:1; regfile `R0`–`R7`:8 each; MMIO `ACT`:8, `DISPLAY`:8; timer `COUNT`, `LIMIT`:8, `EN`, `MATCH`:1; safety `TRIP`, `START_PREV`:1	

The `PC` fields are carried only for observation (pipeline monitor and traces). `CW` bits: `RW`(0) register write, `MR`(1) load, `MW`(2) store, `UA`(3)/`UB`(4) instruction reads `ra`/`rb`, `YI`(5) ALU-Y=imm, `AOP`(7:6) result select, `SB`(8) subtract, `BZ`(9), `BNZ`(10), `JP`(11), `HT`(12). The full per-opcode table is in `isa.pdf`.

3. Stage operation

IF: `I_ADDR=PC`; the ROM word is latched into IF/ID. `NPC` = `REDIRECT` ? `IDEX.IMM` : `PC+1`. **ID**: `IR` is split into `OP`, `RA=IR[11:9]`, `RB=IR[8:6]`, `IMM=IR[7:0]`. The control decoder, both register reads and the hazard check run in parallel. The register file writes on the clock edge, and a WB→ID bypass (`WE∧WA=RA` ? `WD` : `R[RA]`, likewise for `RB`) makes a write-back visible to an ID read in the same cycle. **EX**: operands are forwarded, `Y` = `YI` ? `IMM` : `opB`, the ALU computes, and branch, jump and `HALT` are resolved. `LD`/`ST` pass `opB` through the ALU as the address, and `opA` becomes the store data. **MEM**: one-cycle access to the memory-mapped bus, and `RES` = `MR` ? `D_RDATA` : `ALU`. **WB**: `R[RD]` ← `RES` when `MEMWB.V∧MEMWB.RW`.

4. Forwarding, stall and flush logic (all signals are gate level)

$$
\begin{aligned}
\texttt{USE_A} &= \texttt{IDEX.V} \wedge \texttt{UA}, \quad \texttt{USE_B} = \texttt{IDEX.V} \wedge \texttt{UB} && \text{(only real sources are forwarded)}\\
\texttt{EXM_A} &= \texttt{USE_A} \wedge \texttt{EXMEM.V} \wedge \texttt{EXMEM.RW} \wedge (\texttt{EXMEM.RD} = \texttt{IDEX.RA})\\
\texttt{F_EXMEM_A} &= \texttt{EXM_A} \wedge \neg\texttt{EXMEM.MR} && \text{(a load in EX/MEM is never forwarded)}\\
\texttt{F_MEMWB_A} &= \texttt{USE_A} \wedge \texttt{MEMWB.V} \wedge \texttt{MEMWB.RW} \wedge (\texttt{MEMWB.RD} = \texttt{IDEX.RA}) \wedge \neg\texttt{EXM_A} \text{ (youngest writer wins)}\\
\texttt{opA} &= \texttt{F_EXMEM_A} \ ?\ \texttt{EXMEM.ALU} : (\texttt{F_MEMWB_A} \ ?\ \texttt{MEMWB.RES} : \texttt{IDEX.DA}) && \text{(same for B with RB)}\\
\texttt{STALL_RAW} &= \texttt{IFID.V} \wedge \texttt{IDEX.V} \wedge \texttt{IDEX.MR} \wedge [(\texttt{UA}{\wedge}\texttt{RA}{=}\texttt{IDEX.RA}) \vee (\texttt{UB}{\wedge}\texttt{RB}{=}\texttt{IDEX.RA})]\\
\texttt{REDIRECT} &= \texttt{IDEX.V} \wedge [\texttt{JP} \vee (\texttt{BZ}{\wedge}\texttt{opA}{=}0) \vee (\texttt{BNZ}{\wedge}\texttt{opA}{\neq}0)], \quad \texttt{HALT_EX} = \texttt{IDEX.V} \wedge \texttt{HT}\\
\texttt{FLUSH} &= \texttt{REDIRECT} \vee \texttt{HALT_EX}, \quad \texttt{STALL} = \texttt{STALL_RAW} \wedge \neg\texttt{FLUSH} \text{ (flush has priority)}\\
\texttt{PC_EN} &= \texttt{REDIRECT} \vee \neg(\texttt{STALL_RAW} \vee \texttt{HALT_EX} \vee \texttt{HALTED})\\
\texttt{IFID_EN} &= \neg\texttt{STALL_RAW} \vee \texttt{FLUSH}, \quad \texttt{IFID.V} \leftarrow \neg(\texttt{FLUSH} \vee \texttt{HALTED})\\
\texttt{IDEX.V} &\leftarrow \texttt{IFID.V} \wedge \neg\texttt{STALL_RAW} \wedge \neg\texttt{FLUSH}; \quad \texttt{EXMEM.V} \leftarrow \texttt{IDEX.V}; \quad \texttt{MEMWB.V} \leftarrow \texttt{EXMEM.V}
\end{aligned}
$$

Load-use: PC and IF/ID hold for one cycle, and ID/EX receives a bubble (V=0) while EX/MEM and MEM/WB advance. The consumer then takes the load value from MEM/WB, so it waits exactly one cycle. Distance-2 consumers are forwarded from MEM/WB, and distance-3 consumers read through the WB→ID bypass. A taken branch or jump costs two cycles: the wrong-path entries in IF/ID and ID/EX are invalidated at the edge that loads the target. A load-use stall needs a load in EX and a redirect needs a branch in EX, so they cannot come from the same instruction. The FLUSH term still gives the redirect explicit priority. Architectural writes are gated by V: register file WB_WE=MEMWB.V∧RW, data bus D_WE=EXMEM.V∧MW. MEM never stalls, so each store is presented to the bus for exactly one edge.

5. Reset and HALT

The single RESET input drives the asynchronous CLR of every pipeline, control, I/O, timer and safety register. This gives PC=0, all V=0, HALTED=0, ACT and DISPLAY = 0 (all actuator requests off), a cleared and disabled timer (COUNT, LIMIT, EN and MATCH all 0) and TRIP=0. The register file and RAM are deliberately *not* cleared. When a valid HALT is in EX, HALT_EX freezes the PC, invalidates IF/ID and ID/EX, and sets the HALTED flip-flop (D=1, EN=HALT_EX). After that, IF/ID receives V=0 every cycle and the PC stays frozen. Older instructions in EX/MEM and MEM/WB complete, and the HALT itself drains with no write. Only RESET clears HALTED. A HALT on a squashed path is invalid before it reaches EX and is ignored. The timer and the safety logic keep running while the CPU is stalled or halted.

6. Safety interlock (sub-circuit `safety`)

$$
\begin{aligned}
\texttt{ActualMotorEnable} &= \texttt{REQ} \wedge \texttt{DOOR_CLOSED} \wedge \neg\texttt{TRIP} && (\texttt{REQ} = \texttt{ACT[2]}\text{, i.e. the CPU MOTOR_EN request})\\
\texttt{TRIP}^{+} &= (\texttt{REQ} \wedge \neg\texttt{DOOR_CLOSED}) \vee (\texttt{TRIP} \wedge \neg\texttt{REARM}), \quad \texttt{REARM} = \neg\texttt{REQ} \wedge \texttt{DOOR_CLOSED} \wedge \texttt{START} \wedge \neg\texttt{START_PREV}
\end{aligned}
$$

TRIP and START_PREV (which samples START every cycle) are 1-bit Logisim Registers on the global rising edge. Their clear input is RESET and they have no enable, so they sample every cycle independently of the CPU. TRIP is set at the first edge at which the door is sampled open while the motor request is 1. The output AND gate is combinational, so opening the door drops ActualMotorEnable immediately, without waiting for that edge or for software. Once set, TRIP holds even after the door recloses, so a cleared door sensor alone can never restart the motor. It clears only at an edge where software has removed the request (REQ=0), the door is closed, and a new START 0→1 transition is detected (START_PREV=0, START=1). Set and re-arm need opposite values of REQ, so they can never conflict. The path from ACT[2] to the output is a latch written only by valid MEM-stage stores, so flushed instructions cannot affect it. A faulty program can still request the motor, but can never assert it while the door is open or TRIP=1. The software reads TRIP and the gated output in IN_STATUS[5:4].

7. Clocking and Logisim implementation notes

There is one global rising-edge clock. No clock is gated: all conditional updates use the Register EN input, and the RAM uses its WE input. Clock skew between hierarchy levels in Logisim (at most 2 sub-circuit boundaries, each a 1-unit pin delay) is far smaller than the register clock-to-output delay, so every state element samples pre-edge values. The sub-circuit hierarchy is: main → cpu (control, regfile, hazard, 2×fwd, alu8, muxes) and mmio (timer8, safety, decoders, and8en read multiplexer), with ROM and RAM at the top level so that images are easy to load and inspect. Inside the sub-circuits, every port goes through a short wire stub to a labelled tunnel, so each net name in this document appears as a tunnel label in the schematic.